DuocUC | ONLINE



Guía de aprendizaje

Configuración de tolerancia a fallos y arquitectura de eventos con microservicios en la nube

Exp 3 – Semana 7

Desarrollo Backend III (PBY2203)

Facilitador disciplinar: Ana Karina Villagrán Ibarra

Asesor par: Ignacio Villarroel

Índice

Introducción a la semana....3
Resultado de aprendizaje....4
Conceptos relevantes....4
Preguntas activadoras....4
Actividad....5
Mensajería asincrónica....6
Apache Kafka....8
Introducción a JMS....11
Diseño de una arquitectura orientada a eventos....14
Modelado con Kafka....16
Modelado con JMS....17
Garantías de entrega en sistemas de mensajería....18
¿Cómo manejar la duplicación?....19
Patrones de diseño en una arquitectura de eventos....20
Patrón Saga....21
Event Sourcing....25
¿Cuándo usar Event Sourcing?....28
Proyecto con Kafka y JMS....29
Cierre de la semana....30
Referencias....31
Lecturas de la semana....31
Apuntes....32

Introducción a la semana



Esta semana te enfocarás en reforzar conceptos fundamentales para el desarrollo de sistemas backend resilientes y escalables en la nube. Profundizarás en cómo las arquitecturas orientadas a eventos y los mecanismos de tolerancia a fallos son esenciales para mantener la estabilidad y eficiencia en sistemas distribuidos. A través de este aprendizaje, comprenderás cómo tecnologías como Kafka y JMS pueden integrarse para fortalecer la resiliencia y el procesamiento asíncrono de eventos.

Posteriormente, explorarás cómo configurar arquitecturas orientadas a eventos, integrando componentes asíncronos para procesar datos de manera eficiente y escalable. También te centrarás en asegurar que los microservicios puedan operar de forma confiable ante interrupciones, siguiendo buenas prácticas de tolerancia a fallos y diseño de sistemas distribuidos.

Este aprendizaje te permitirá consolidar habilidades clave para construir sistemas robustos y altamente disponibles, capaces de operar en entornos complejos. Al finalizar, estarás mejor preparado para diseñar soluciones backend avanzadas que combinen seguridad, resiliencia y capacidad de adaptación a los retos del desarrollo cloud moderno.

Resultado de aprendizaje

El estudiante será capaz de:

RA3. Desarrolla componentes backend avanzados utilizando tecnologías cloud que permitan disponibilizar sistemas resistentes a fallos, de acuerdo a las buenas prácticas de la industria.

Indicador de logro:

IL7. Configura mecanismos de seguridad y tolerancia a fallos utilizando Resilience4j, garantizando la resiliencia del sistema ante fallos.

IL8. Integra componentes asíncronos de mensajería utilizando Kafka y JMS, para implementar una arquitectura orientada a eventos.

Conceptos relevantes

Mensajería Asincrónica	Kafka	JMS
Event Sourcing	Patrón Saga	Transacciones distribuidas

Preguntas activadoras

- ¿Cuál es la importancia del diseño de una arquitectura orientada a eventos para garantizar la flexibilidad y escalabilidad de los sistemas distribuidos?
- ¿Qué aspectos clave debes considerar al modelar sistemas basados en eventos con herramientas como Kafka o JMS?
- ¿Cómo influye la elección entre patrones como el Event Sourcing y el Patrón Saga en la implementación de arquitecturas orientadas a eventos?

Actividad

Descripción de la actividad

Durante esta séptima semana, desarrollarás una actividad formativa grupal llamada "Configurando tolerancia a fallos y arquitectura de eventos con microservicios en la nube", en la que deberás fortalecer la resiliencia de microservicios en la nube implementando mecanismos de tolerancia a fallos con Resilience4j. Además, integrarás componentes asíncronos de mensajería utilizando Kafka y JMS, habilitando la arquitectura para procesar eventos de manera eficiente y escalable.

Mensajería asincrónica

Para entender por qué una comunicación asíncrona podría mejorar arquitecturas de microservicios, analicemos un caso:

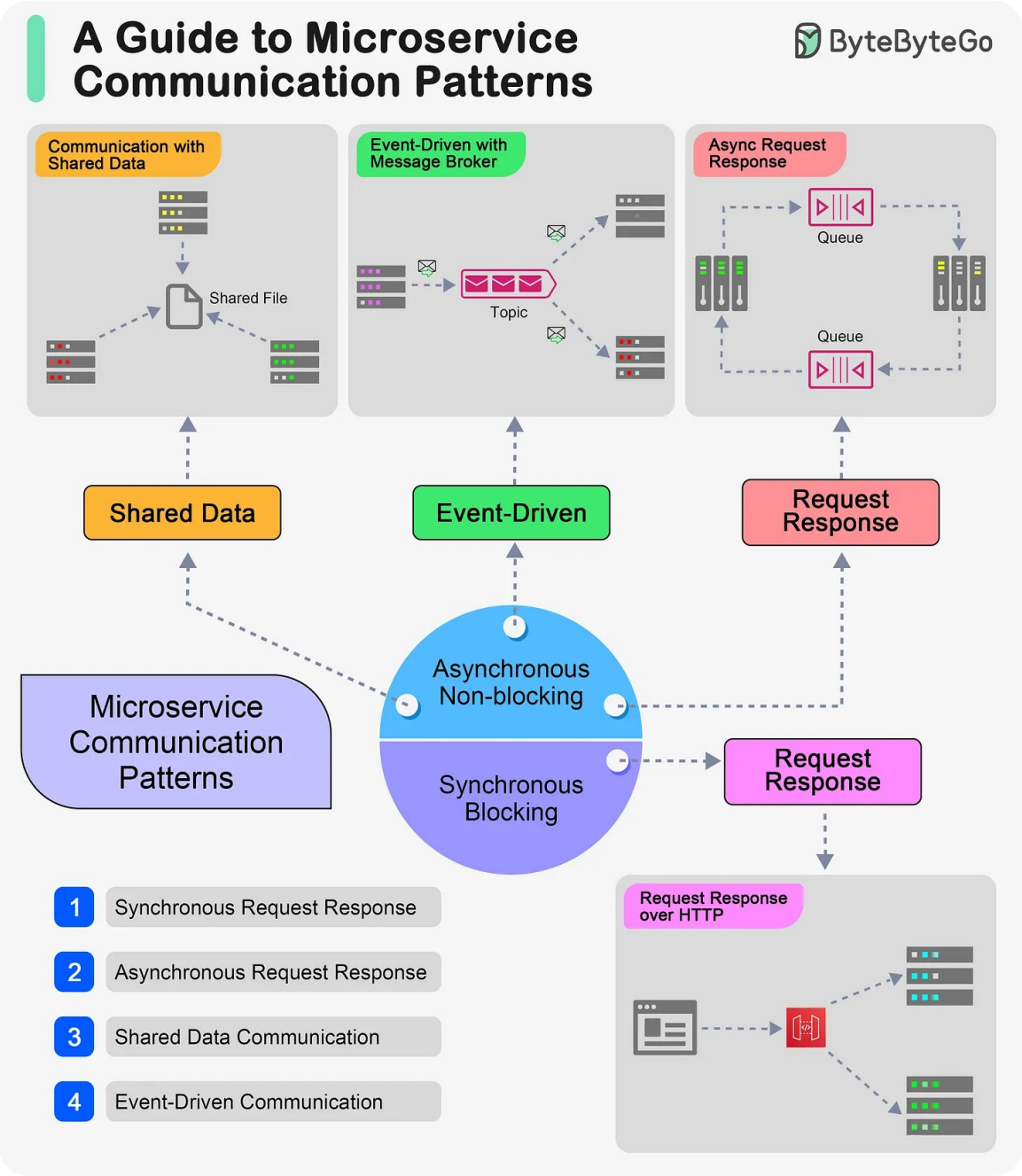


Imagina que en una cafetería se reciben pedidos en el mostrador. Si el barista hace un café por vez y entrega cada pedido antes de recibir otro, el sistema será lento y las personas en la fila se frustrarán. Ahora, si el barista recibiera los pedidos, anotando cada uno en una lista y los prepara en el orden en que llegaron, mientras los clientes esperan cómodamente, mejorará considerablemente la experiencia.

Aquí, la lista es como un sistema de mensajería, donde los pedidos (mensajes) se almacenan temporalmente y son procesados según la disponibilidad del barista (servicio).

Figura 1

Infografía de patrones de comunicación de microservicios.



Nota. Esta imagen muestra esquemas de patrones de comunicación de microservicios, señalando la respuesta de solicitud sincrónica, respuesta de solicitud asincrónica, comunicación de datos compartidos y comunicación basada en eventos. Bytebytego (2024).

A Crash Course on Microservice Communication Patterns. https://blog.bytebytego.com/p/a-crash-course-on-microservice-communication

Este enfoque tiene varios beneficios: si un cliente se demora en recoger su pedido (equivalente a un fallo temporal en un servicio), los demás pedidos pueden seguir siendo preparados. Además, el barista puede organizar mejor su tiempo y procesar los pedidos de forma eficiente, incluso si llegan muchos al mismo tiempo. Esto es similar a cómo los sistemas de mensajería permiten que los servicios en arquitecturas distribuidas trabajen de manera flexible y eficiente, manejando altos volúmenes de solicitudes y garantizando que ninguna se pierda.

En el universo de plataformas de mensajería asíncrona, encontramos soluciones tales como Kafka, RabbitMQ, PubSub, entre otras muchas opciones. Cada una entrega diferentes alternativas de cómo gestionar la mensajería en tu ecosistema de aplicaciones. Profundicemos en Kafka, que es uno de los proyectos más importante para este tópico.

Apache Kafka



Apache Kafka es una plataforma distribuida de mensajería y procesamiento de datos en tiempo real. Fue diseñada inicialmente por LinkedIn y posteriormente donada a la Apache Software Foundation. Kafka es ideal para construir sistemas que manejan flujos de datos grandes y necesitan procesarlos de forma eficiente, fiable y escalable.

En esencia, Kafka actúa como un intermediario de mensajes que permite que diferentes sistemas se comuniquen entre sí. Funciona como un "bus" de datos donde los productores (que generan datos) envían mensajes, y los consumidores (que procesan esos datos) los leen de forma independiente.

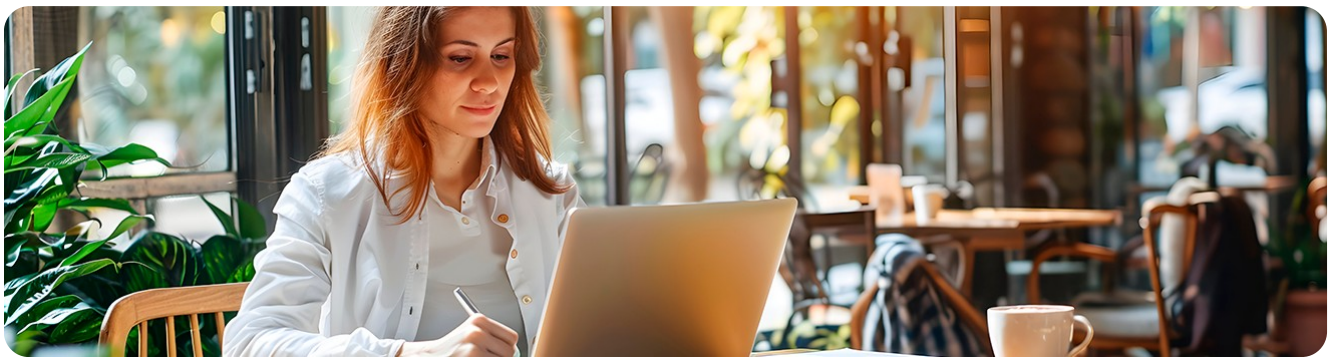
Kafka está diseñado para ser rápido, escalable y duradero, y es ampliamente utilizado en sistemas distribuidos para manejar eventos, transacciones, monitoreo de logs, análisis de datos en tiempo real y más.

Links de Interés

Apache Kafka es una plataforma de transmisión de eventos distribuida de código abierto utilizada para canalizaciones de datos de alto rendimiento, análisis de transmisión, integración de datos y aplicaciones de misión crítica. Su sitio web: https://kafka.apache.org/

Conceptos clave

Imagina que estás organizando una fiesta y necesitas coordinar varias actividades, como enviar invitaciones, preparar la comida y decorar el lugar. En este ejemplo, un **evento** es cualquier acción que ocurre, como "invitación enviada" o "comida preparada". Estos eventos son como mensajes que indican que algo ha sucedido y necesitan ser **procesados**.



En esta fiesta, tú actúas como el productor, que genera los eventos. Por ejemplo, cada vez que envías una invitación, produces un evento "invitación enviada". Estos eventos se colocan en un tópico, que sería como una bandeja central donde todos los mensajes relacionados con la fiesta se almacenan. Por ejemplo, podrías tener un tópico llamado "preparativos-fiesta".

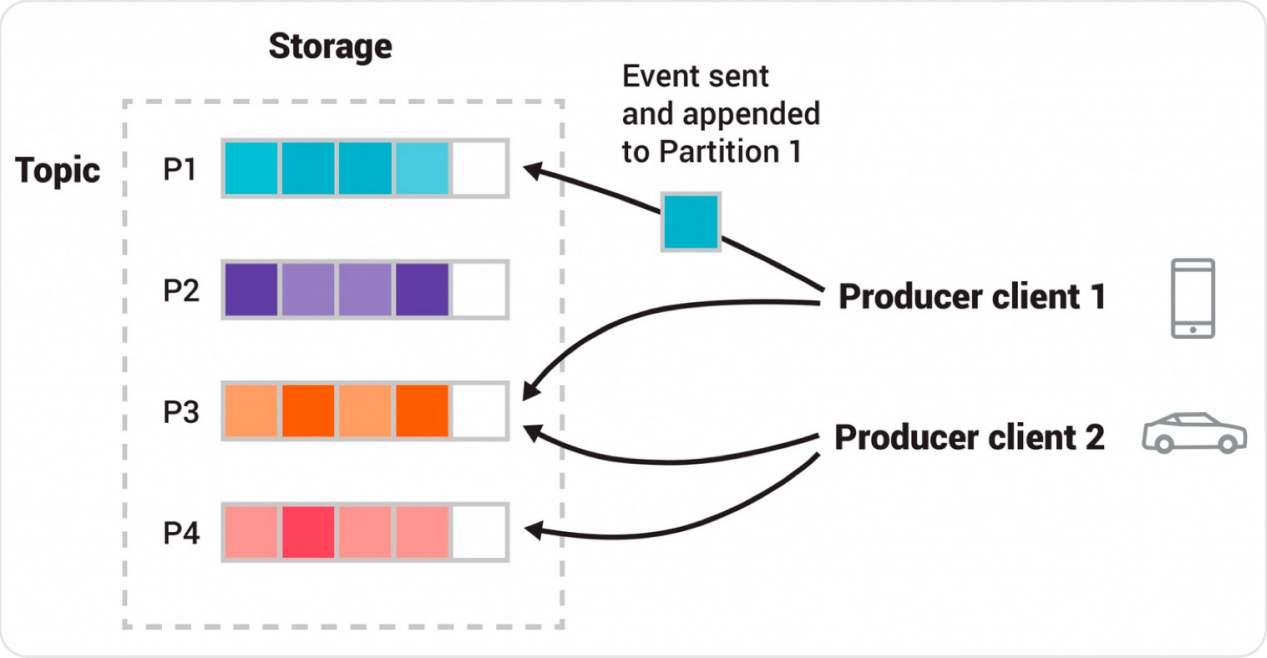
Dentro del tópico, los eventos se dividen en **particiones**. Cada partición es como una sección separada de la bandeja que organiza los eventos de manera más eficiente. Por ejemplo, una partición podría almacenar todos los eventos relacionados con invitaciones, mientras que otra partición guarda los eventos de decoración. Esto permite que varias personas trabajen al mismo tiempo.

Por otro lado, los **consumidores** son las personas que toman estos eventos desde las particiones para procesarlos. Por ejemplo, alguien puede ser el consumidor encargado de leer los eventos de "invitaciones enviadas" para confirmar que todos los invitados han recibido la suya, mientras que otro consumidor se encarga de los eventos de "decoración terminada" para asegurarse de que el salón está listo.

En este sistema, tú produces los eventos, el tópico organiza los mensajes, las particiones distribuyen el trabajo, y los consumidores procesan los eventos, todo de forma ordenada y eficiente. Kafka funciona de manera similar para sistemas informáticos, permitiendo que diferentes partes de una aplicación trabajen en equipo sin depender directamente unas de otras.

Figura 2

Representación conceptual de Kafka



Nota. Esta imagen muestra cómo funciona Apache Kafka organizando eventos en un tópico dividido en múltiples particiones. Kafka. (s.f.). Kafka Introduction. Apache Kafka. https://kafka.apache.org/intro

Introducción a JMS



Java Message Service (JMS) es una API estándar de Java que te permite enviar, recibir y procesar mensajes de manera asíncrona en sistemas distribuidos. En lugar de conectar directamente los componentes de tu sistema, JMS actúa como un intermediario, desacoplando los productores (quienes envían mensajes) de los consumidores (quienes procesan los mensajes). Esto te permite construir sistemas más flexibles y resilientes. Con JMS, puedes usar dos modelos de mensajería principales: el modelo de colas (Point-to-Point), donde un mensaje es procesado por un único consumidor, o el modelo de publicación-suscripción (Publish-Subscribe), donde varios consumidores pueden recibir el mismo mensaje.

Links de Interés

Para conocer más especificaciones, puedes visitar el siguiente link:
https://jcp.org/en/jsr/detail?id=343

A continuación, te mostramos las diferencias clave entre Kafka y JMS, ya que, con solo una explicación general, pareciera que son lo mismo.

Tabla 1

Diferencias entre Kafka y JMS

Aspecto	**Kafka**	**JMS**
Propósito general	Procesamiento de eventos y flujos de datos en tiempo real	Comunicación transaccional e integración de sistemas empresariales
Modelo de mensajería	Publish-Subscribe (Topicos) con almacenamiento persistente	Point-to-Point (colas) y Publish-Subscribe (tópicos)
Retención de mensajes	Los mensajes se almacenan durante un tiempo configurado (por defecto, días)	Los mensajes suelen eliminarse después de ser consumidos
Protocolos soportados	Protocolo propietario optimizado para alto rendimiento.	Compatible con el estándar Java Message Service (JMS)
Escalabilidad	Altamente escalable; maneja millones de mensajes por segundo	Escalabilidad limitada, dependiendo del proveedor (como ActiveMQ)
Casos de uso	Procesamiento de eventos, análisis en tiempo real, arquitecturas modernas	Comunicación punto a punto, enrutamiento de mensajes, sistemas empresariales
Desacoplamiento	Los consumidores leen a su propio ritmo, incluso múltiples veces	Los mensajes son entregados directamente y procesados por un único consumidor (en colas)
Tecnología base	Diseñado para sistemas distribuidos, con almacenamiento basado en logs	Depende del proveedor JMS (por ejemplo, ActiveMQ, RabbitMQ)

Links de Interés

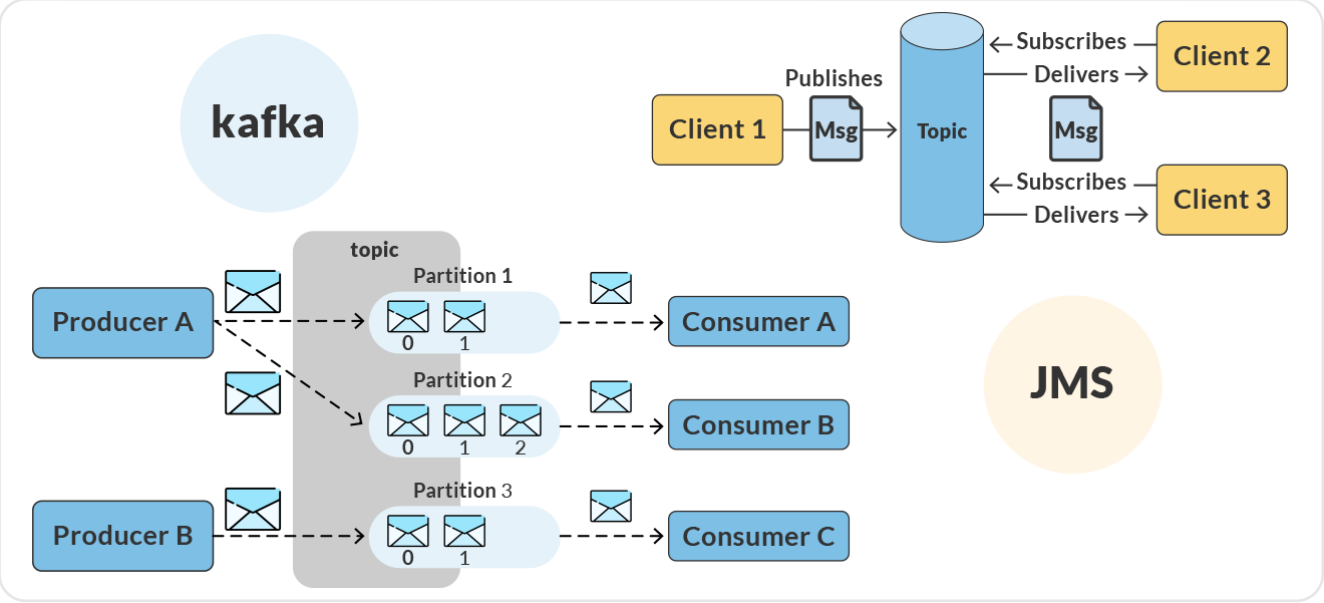
Si te interesa profundizar en esta comparación de JMS y Apache Kafka, te recomendamos visitar el siguiente enlace:

https://www.kai-waehner.de/blog/2022/05/12/comparison-jms-api-message-broker-mq-vs-apache-kafka/

En resumen, tanto Kafka como JMS son formas de establecer comunicación asíncrona en plataformas tecnológicas, y dependerá de las necesidades y recursos del proyecto determinar que solución utilizar.

Figura 3

Diferencia entre Kafka y JMS



Nota. La imagen muestra Apache Kafka con implementaciones populares de estándares de mensajería Java. Vichugovae, A. (2022). *Brokers Apache Kafka vs JMS: 3 diferencias principales.* Bigdataschool. https://bigdataschool.ru/blog/kafka-vs-jms-brokers.html

Diseño de una arquitectura orientada a eventos

Un evento es una representación de algo que ha sucedido en un sistema, como un cambio de estado o una acción específica. En sistemas distribuidos, los eventos son fundamentales para comunicar información entre diferentes servicios de manera eficiente y asíncrona, permitiendo que cada componente reaccione a los cambios sin depender directamente de los demás.

Piensa en la organización de una fiesta, como se explicó anteriormente. Cada acción, como "invitación enviada" o "decoración terminada", es un evento que puede ser gestionado por diferentes personas (o servicios) según su responsabilidad. De manera similar, en un sistema distribuido, un evento como "pedido realizado" puede ser procesado por múltiples servicios: el de facturación para generar una factura, el de inventario para descontar productos y el de notificaciones para enviar un correo al cliente. Este enfoque desacopla los servicios, mejora la escalabilidad y permite manejar grandes volúmenes de datos sin interrupciones.

Figura 4

Componentes de una arquitectura orientada a eventos



Nota. En esta imagen se observa la arquitectura orientada a eventos, cuyos sistemas se dividen en dos componentes principales: los productores de eventos y consumidores de eventos. Coder, D. (2023). *Introducción a la arquitectura orientada a eventos.* Medium.

https://medium.com/@diego.coder/introducci%C3%B3n-a-la-arquitectura-orientada-a-eventos-a532c71c9945

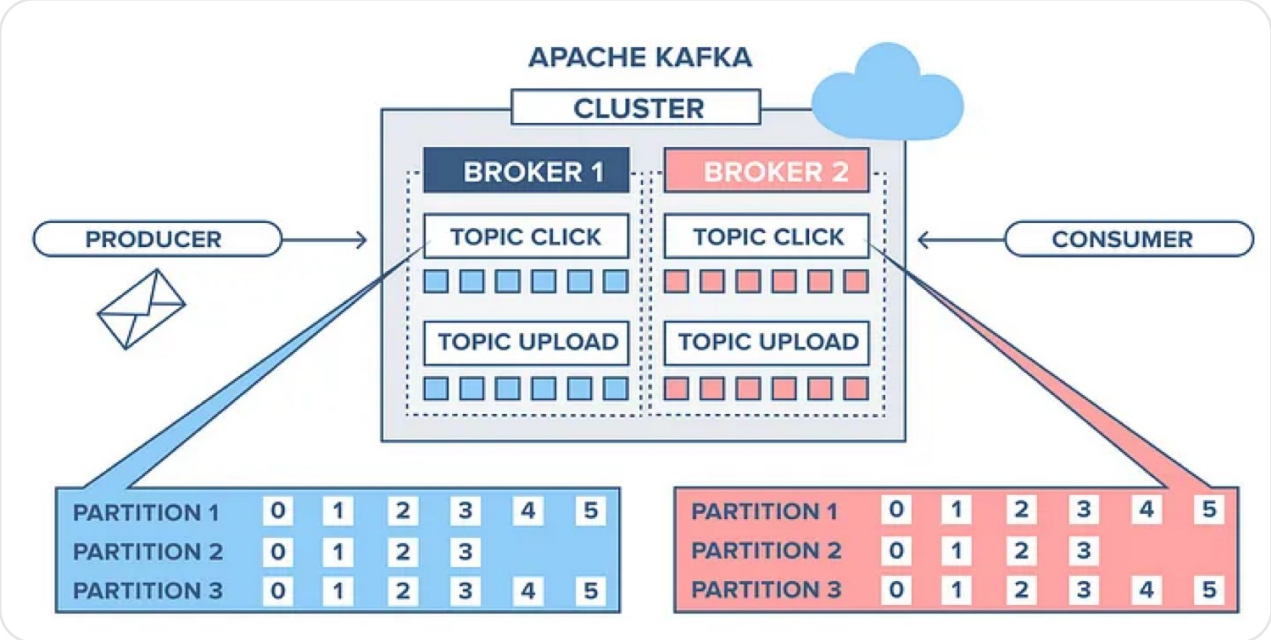
Cuando modelas sistemas basados en eventos, necesitas una infraestructura que gestione los eventos eficientemente. Herramientas como Kafka o JMS facilitan este proceso, permitiéndote organizar, distribuir y procesar eventos entre los diferentes componentes del sistema. Revisemos a continuación, el modelado con cada una de estas herramientas.

Modelado con Kafka

Su arquitectura es la siguiente:

Figura 5

Arquitectura de Apache Kafka



Nota. En la imagen se muestra los conceptos claves de la arquitectura: Productores, tópicos, particiones, broker, consumidores y grupos de consumidores. Coder, D. (2023). *Fundamentos de Apache.* Medium. https://medium.com/@diego.coder/introducci%C3%B3n-a-apache-kafka-d1118be9d632

- **Tópicos:** piensa en ellos como bandejas donde se almacenan los eventos de un tipo específico, como "pedidos". Cada tópico contiene los eventos organizados en orden de llegada.
- **Particiones:** los tópicos se dividen en particiones, lo que permite que múltiples consumidores trabajen simultáneamente en paralelo. Por ejemplo, en un tópico "pedidos", una partición puede procesar los eventos relacionados con un tipo de producto.
- **Productores y consumidores:** los productores son los servicios que generan eventos (como el servicio de pedidos), y los consumidores son los servicios que procesan esos eventos (como facturación o notificaciones). Kafka permite que múltiples consumidores lean el mismo evento desde diferentes particiones.

Con Kafka, los eventos se almacenan de forma persistente por un tiempo configurado, lo que permite reprocesarlos o manejarlos en caso de fallos.

Modelado con JMS

- Colas: los eventos se envían a colas, donde cada mensaje es procesado por un único consumidor. Esto es ideal para tareas punto a punto, como procesar transacciones bancarias.
- Tópicos: similar a Kafka, pero en JMS los tópicos permiten que múltiples consumidores reciban el mismo mensaje, lo que es útil en sistemas donde diferentes servicios necesitan reaccionar al mismo evento.
- Proveedores JMS: herramientas como ActiveMQ implementan JMS, ofreciendo funcionalidades como persistencia y enrutamiento de mensajes.

A diferencia de Kafka, en JMS los mensajes suelen eliminarse después de ser procesados, y su enfoque es más adecuado para sistemas empresariales transaccionales.

¿Cómo saber cuál implementar?

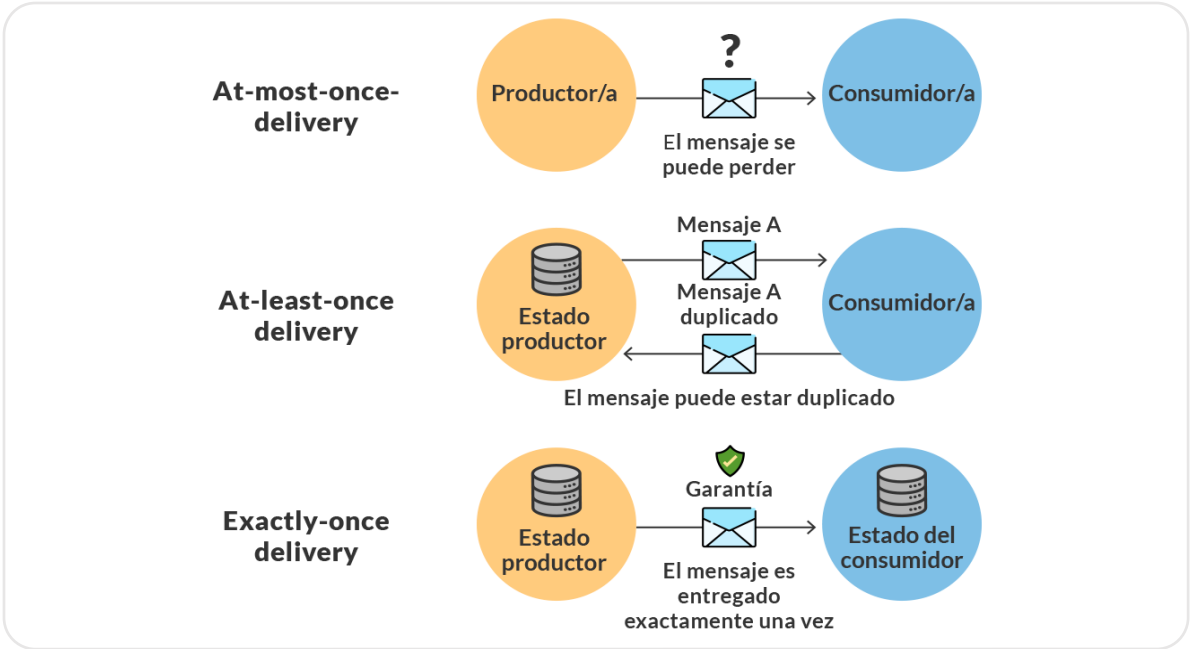
Imagina que estás organizando la fiesta y decides usar Kafka o JMS para gestionar las tareas. En Kafka, todos los eventos relacionados con la fiesta (invitaciones, decoración, comida) se almacenan en un tópico llamado "preparativos". Cada partición dentro del tópico organiza un tipo específico de tarea, permitiendo que múltiples personas (consumidores) trabajen en paralelo. Por otro lado, con JMS, podrías enviar cada tarea (como "decorar la sala") a una cola específica, y un único consumidor (por ejemplo, el decorador) se encargaría de procesarla.

Garantías de entrega en sistemas de mensajería

Cuando trabajas con sistemas de mensajería, es fundamental entender cómo se garantizan las entregas de los mensajes entre productores y consumidores. Dependiendo de la configuración y el enfoque, puedes manejar los mensajes de diferentes maneras para equilibrar confiabilidad y eficiencia. Estas garantías de entrega son clave para asegurarte de que los datos lleguen correctamente al destino. Dependiendo de cómo se comporta el sistema cuando ocurren fallas, se obtienen varios tipos de semántica de mensajería:

Figura 6

Tipos de entrega



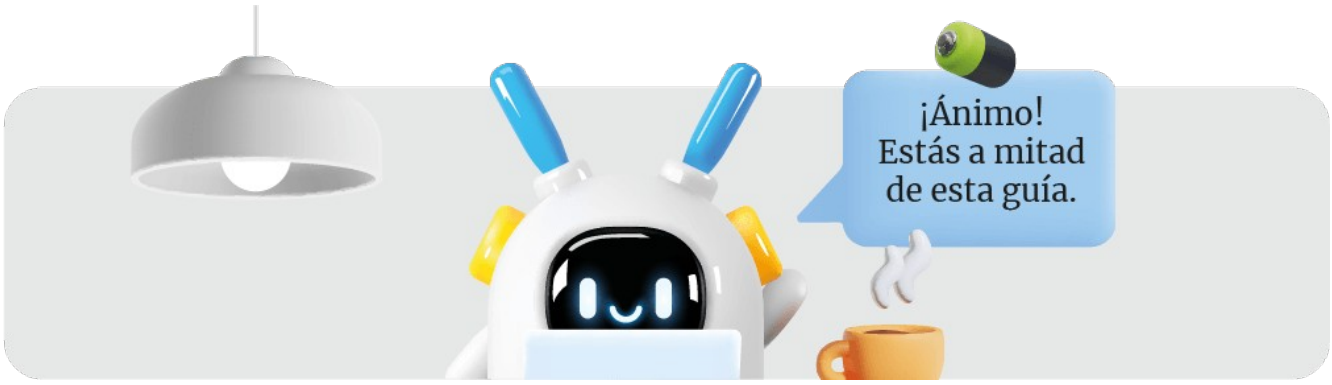
*Nota. En la imagen se muestra tipos de g*arantías de entrega en sistemas de mensajería. Byersm, P. (2024). *Achieving exactly-once delivery with Ably. Ably.* https://ably.com/blog/achieving-exactly-once-message-processing-with-ably

- ***At-most-once* (como máximo una vez):** con este enfoque, los mensajes se entregan solo una vez, pero existe el riesgo de que algunos se pierdan si ocurre un error en el envío o procesamiento. Es como enviar un mensaje de texto sin esperar confirmación; puede que llegue o puede que no. Este método es útil cuando prefieres evitar duplicaciones, aunque pierdas algún mensaje, por ejemplo, en aplicaciones de monitoreo donde no necesitas cada actualización.
- ***At-least-once* (al menos una vez):** aquí, los mensajes se entregan una o más veces, asegurando que ninguno se pierda. Sin embargo, podrías recibir duplicados porque el sistema puede reenviar un mensaje si no recibe una confirmación del consumidor. Este enfoque es ideal para aplicaciones críticas, como pedidos en línea, donde prefieres procesar un pedido dos veces que perderlo.
- ***Exactly-once* (exactamente una vez):** este es el nivel más confiable y asegura que cada mensaje se entregue y procese solo una vez, incluso en caso de fallos. Es como enviar un paquete con firma obligatoria: solo se considera "entregado" cuando hay una confirmación única. Este enfoque es más complejo y consume más recursos, pero es esencial para sistemas financieros o de pagos.

¿Cómo manejar la duplicación?

En sistemas distribuidos, es común enfrentarse a la duplicación o pérdida de mensajes debido a fallos en la red, errores en los servicios o reintentos de entrega. Para manejar estos casos, puedes implementar las siguientes estrategias:

- Uso de identificadores únicos: asegúrate de que cada mensaje tenga un identificador único (como un UUID). De esta forma, los consumidores pueden detectar duplicados y procesar solo los mensajes que aún no hayan visto.
- Persistencia en el consumidor: almacena un registro de los mensajes procesados. Antes de procesar un nuevo mensaje, verifica si ya ha sido registrado y, si es así, ignóralo.
- Reintentos controlados: configura un número máximo de reintentos en el sistema de mensajería para evitar que un mensaje se envíe infinitamente en caso de fallo.
- Desduplicación en el productor o sistema central: si la duplicación ocurre a nivel del sistema de mensajería, puedes implementar un mecanismo de desduplicación en el intermediario (por ejemplo, Kafka o RabbitMQ) para filtrar los mensajes repetidos antes de enviarlos al consumidor.



Patrones de diseño en una arquitectura de eventos

Como ya sabes, en el desarrollo de software moderno, las arquitecturas basadas en eventos han ganado popularidad debido a su capacidad para manejar sistemas distribuidos y escalables de manera eficiente. Estas arquitecturas giran en torno al concepto de eventos, que son representaciones de cambios o acciones que ocurren dentro de un sistema. Los patrones de diseño en una arquitectura de eventos proporcionan soluciones probadas para problemas comunes al trabajar con sistemas distribuidos, asegurando consistencia, resiliencia y escalabilidad.

Estos patrones permiten manejar flujos complejos, sincronización entre servicios y el procesamiento de datos de manera asincrónica. Entre los patrones más destacados dentro de este enfoque se encuentran el Patrón Saga y el Event Sourcing, los cuales abordan desafíos específicos de la coordinación de transacciones distribuidas y la gestión del estado de los sistemas.

Patrón Saga

Cuando trabajas con microservicios, una de las mayores dificultades es manejar transacciones distribuidas. A diferencia de un sistema monolítico, donde una transacción se puede gestionar fácilmente dentro de una única base de datos, en microservicios, los datos suelen estar distribuidos en múltiples servicios. Esto hace que garantizar la consistencia sea un desafío.

El Patrón Saga es un patrón de diseño que permite gestionar transacciones distribuidas dividiéndolas en una serie de pasos independientes, donde cada servicio realiza una acción específica y notifica su resultado al siguiente. Cada paso de la saga es una transacción local en el servicio correspondiente.

Existen dos formas principales de implementar el Patrón Saga:

Coreografía (Choreography)

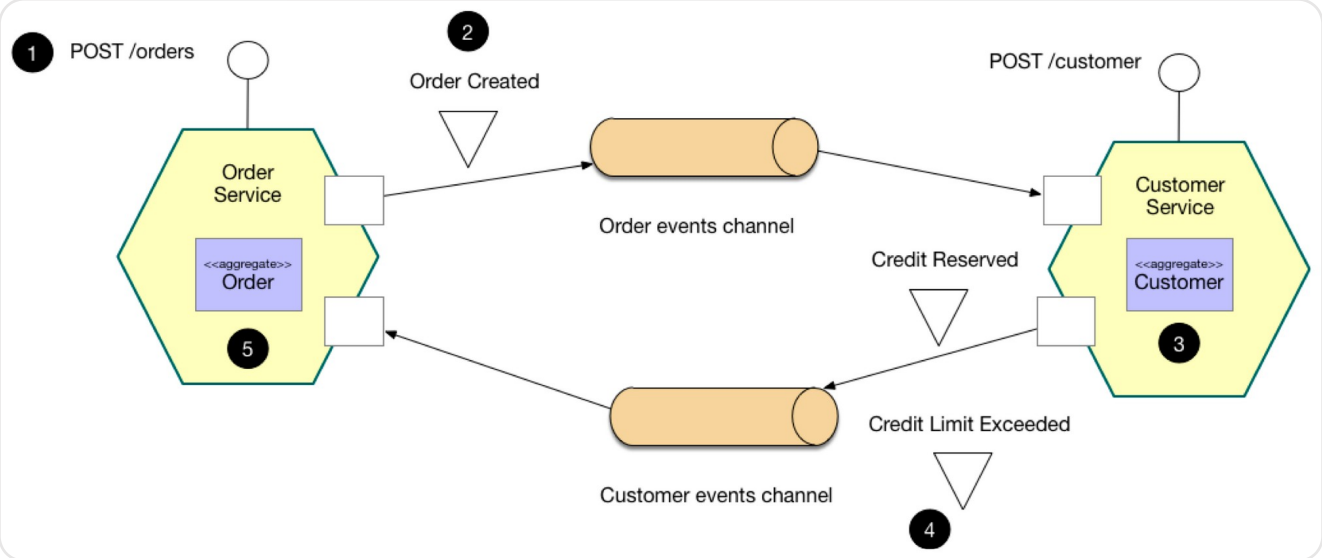
En este enfoque, los servicios se comunican entre sí mediante eventos. Cuando un servicio completa su acción, publica un evento que activa el siguiente paso.

Ejemplo: en un sistema de comercio electrónico, cuando un cliente realiza un pedido, el servicio de "pedidos" publica un evento pedido creado. El servicio de "inventario" consume ese evento, verifica el stock y publica otro evento inventario reservado, que activa al servicio de "pago".

En la Figura 7, una aplicación de comercio electrónico que utiliza este enfoque crearía un pedido utilizando una saga basada en coreografía que consiste en los siguientes pasos:

Figura 7

Patrón Saga: Coreografía



Nota. Esta imagen representa una saga coreografiada utilizada en una aplicación de comercio electrónico para gestionar la creación de pedidos y la validación del crédito disponible de un cliente. Richardson, C. (s.f.). *Managing data consistency in a microservice architecture using Sagas - Implementing a choreography-based saga.* Microservices. https://microservices.io/post/sagas/2019/08/15/developing-sagas-part-3.html

1. El Order Service recibe una solicitud POST/orders y crea un pedido en estado PENDING (pendiente).
2. Luego, emite un evento llamado Order Created (Pedido creado).
3. El manejador de eventos del Customer Service intenta reservar crédito.
4. A continuación, emite un evento indicando el resultado (éxito o fallo en la reserva del crédito).
5. El manejador de eventos del Order Service aprueba o rechaza el pedido dependiendo del resultado recibido.

Orquestación (Orchestration)

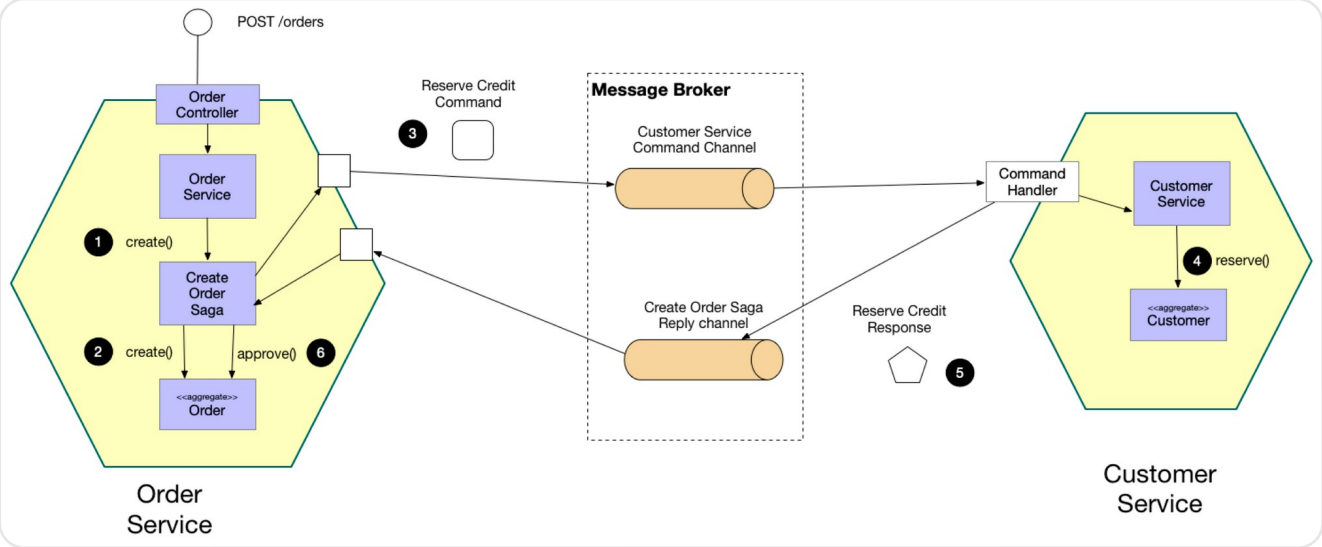
Aquí, un servicio central (llamado orquestador) coordina toda la transacción. El orquestador envía comandos a cada servicio y espera respuestas antes de avanzar al siguiente paso.

Ejemplo: un orquestador envía un comando al servicio de "inventario" para reservar productos. Si el inventario lo confirma, el orquestador envía un comando al servicio de "pago" para procesar la transacción.

En la Figura 8, una aplicación de comercio electrónico que utiliza este enfoque crearía un pedido utilizando una saga basada en orquestación que consiste en los siguientes pasos:

Figura 8

Patrón Saga: Orquestación



Nota. Esta imagen ilustra cómo funciona una saga orquestada para gestionar transacciones distribuidas en un sistema de comercio electrónico. Richardson, C.(s.f.). Pattern: Saga. Microservices. https://microservices.io/patterns/data/saga.html

1. El Order Service recibe una solicitud POST /orders y crea el orquestador de la saga Create Order.
2. El orquestador de la saga crea un pedido en estado PENDING (pendiente).
3. Luego, envía un comando Reserve Credit (Reservar crédito) al Customer Service.
4. El Customer Service intenta reservar el crédito del cliente.
5. A continuación, envía un mensaje de respuesta indicando el resultado (éxito o fallo en la reserva del crédito).
6. El orquestador de la saga aprueba o rechaza el pedido dependiendo del resultado recibido.

El tipo de implementación de Saga influye en la tecnología elegida porque cada patrón tiene necesidades específicas. En una coreografía, donde los servicios trabajan de manera descentralizada reaccionando a eventos, Kafka es ideal por su alta escalabilidad, manejo eficiente de eventos y persistencia, mientras que JMS es útil en sistemas empresariales con menor volumen de eventos y mayor necesidad de consistencia transaccional.

En una orquestación, donde un orquestador central coordina las acciones, Kafka es adecuado para manejar grandes volúmenes de datos con canales de comandos y respuestas, mientras que JMS es más sencillo para comunicación punto a punto en sistemas con flujos transaccionales bien definidos. La elección depende de la escala, complejidad y requisitos del sistema.

Links de interés

Puedes encontrar más información sobre el Patrón Saga en:
https://microservices.io/patterns/data/saga.html

Event Sourcing

Event Sourcing es un patrón arquitectónico en el que los eventos que ocurren en un sistema se almacenan como la fuente única de verdad. En lugar de guardar únicamente el estado actual de un objeto (por ejemplo, el saldo de una cuenta bancaria), se registra cada cambio o acción que afecta a ese objeto en forma de eventos. El estado actual se calcula aplicando todos los eventos en el orden en que ocurrieron.

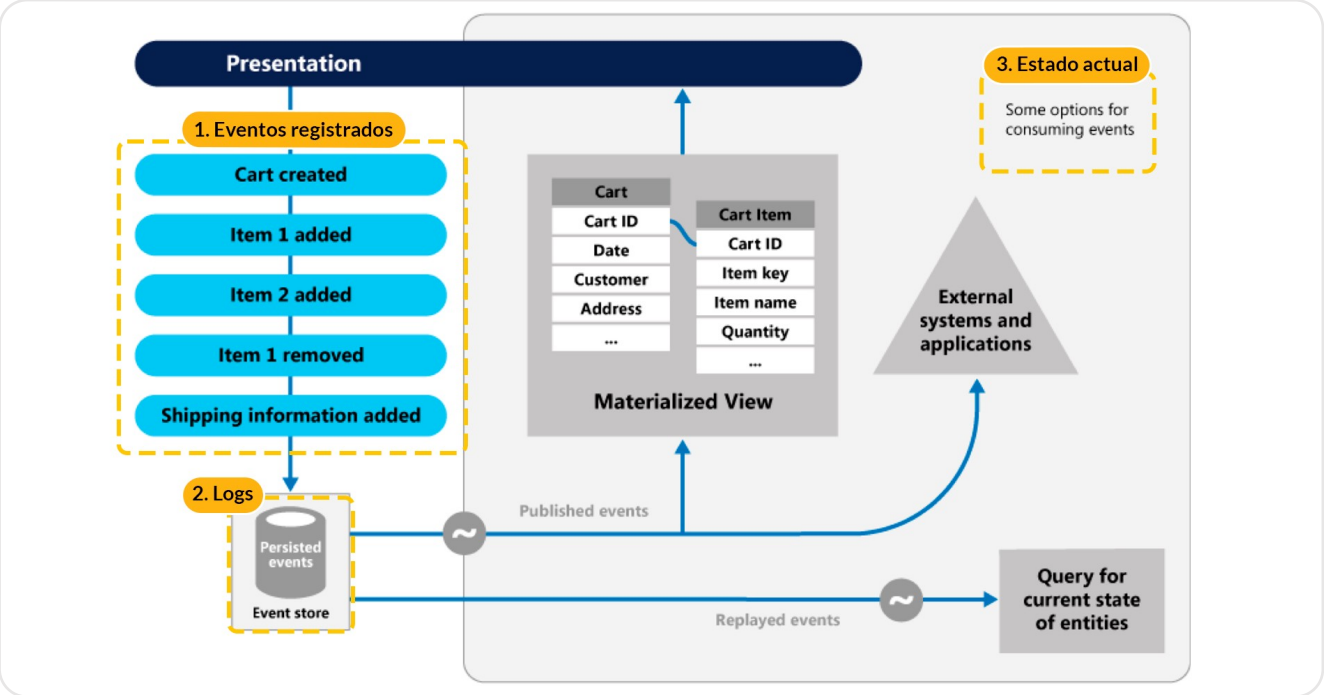
Este patrón es especialmente útil en arquitecturas basadas en eventos, ya que no solo captura lo que sucede, sino también cómo y cuándo sucede. Esto permite no solo reconstruir el estado actual, sino también realizar análisis históricos y auditar todas las acciones realizadas en el sistema.

¿Cómo funciona?

- Registro de eventos: cada vez que ocurre un cambio en el sistema, se genera un evento que describe esa acción (por ejemplo, "Depósito realizado", "Producto agregado al carrito"). Este evento es inmutable y se almacena en un log de eventos.
- Log de eventos: todos los eventos se guardan en un log centralizado y ordenado. Este log es el registro completo de la historia del sistema.
- Reconstrucción del estado: el estado actual de un objeto no se almacena directamente. En su lugar, se reconstruye aplicando todos los eventos en el orden en que ocurrieron.
- Proyección de vistas: puedes generar "proyecciones" del estado actual a partir de los eventos, optimizadas para consulta o análisis, sin afectar el log de eventos original.

Figura 9

Introducción al patrón



Nota. La imagen incluye algunas de las opciones de uso del flujo de eventos, como la creación de una vista materializada, la integración de eventos con aplicaciones, sistemas externos y la reproducción de eventos para crear proyecciones del estado actual de entidades específicas. Microsoft. (s.f.). Patrón Event Sourcing. Learn.microsoft. https://learn.microsoft.com/es-es/azure/architecture/patterns/event-sourcing

Links de interés

Puedes encontrar más información sobre Event Sourcing en: https://learn.microsoft.com/es-es/azure/architecture/patterns/event-sourcing

Caso: Secuencia funcional de Event sourcing.

Imagina que estás construyendo una aplicación bancaria. En lugar de guardar el saldo actual de una cuenta, utilizas Event Sourcing para registrar los eventos:

Eventos registrados:

- Depósito de $100
- "Retiro de $50
- "Depósito de $200

Log de eventos

- [Evento 1: Depósito de $100]
- [Evento 2: Retiro de $50]
- [Evento 3: Depósito de $200]

Estado actual: para calcular el saldo actual, aplicas cada evento en secuencia:

- Saldo inicial: $0.
- Después del Evento 1: $100.
- Después del Evento 2: $50.
- Después del Evento 3: $250.

Las ventajas del evento sourcing son su auditabilidad, reconstrucción de estado, flexibilidad y desacoplamiento. Por otro lado, debes considerar sus desafíos tales como complejidad a la hora de implementar, reprocesar eventos donde existen miles de registros y el manejo de datos sensibles, ya que si lo mensajes tienen información confidencial podría comprometer a la organización.

¿Cuándo usar Event Sourcing?

- **Necesidad de trazabilidad:** si necesitas un historial completo de todos los cambios, como en sistemas financieros, de salud o legales.
- **Escenarios basados en eventos:** si tu sistema necesita reaccionar a eventos en tiempo real y mantener un registro confiable de ellos.
- **Reconstrucción de estado:** cuando necesitas la capacidad de recrear el estado actual o histórico del sistema.
- **Desacoplamiento:** en arquitecturas distribuidas donde los servicios no deben depender directamente unos de otros.

Revisa una tabla resumen “cuando usar” cada patrón.

Tabla 2

Cuándo usar el patrón Event sourcing o el patrón Saga

Usar Event sourcing sí…	**Usar Saga sí…**
Necesitas un historial completo de eventos del sistema	Necesitas coordinar acciones entre múltiples servicios
Quieres reconstruir estados o realizar auditorías	Quieres manejar transacciones distribuidas con consistencia
Trabajas en una arquitectura orientada a eventos	Tienes operaciones que necesitan pasos compensatorios
Manejas grandes volúmenes de datos con proyecciones personalizadas	Tus servicios dependen de un flujo secuencial de operaciones

Proyecto con Kafka y JMS

Puedes revisar estos dos proyectos que te permitirán ver una implementación con Kafka y JMS (Usando ActiveMQ). El objetivo es crear un proyecto que se logre integrar con Kafka o JMS.

Links de Interés

- Ejemplo de Spring Boot con Kafka:

  https://github.com/spring-cloud/spring-cloud-stream-samples/tree/main/kafka-batch-sample

- Ejemplo de Spring Boot con ActiveMQ:

  https://github.com/spring-guides/gs-messaging-jms

Cierre de la semana



Durante estos días, profundizaste en conceptos clave como la mensajería asíncrona, explorando cómo herramientas como Kafka y JMS permiten gestionar eventos y garantizar la comunicación eficiente entre servicios. Aprendiste a implementar garantías de entrega, manejar duplicaciones y procesar mensajes de forma confiable para construir sistemas robustos.

También abordaste patrones avanzados como Saga para gestionar transacciones distribuidas, tanto en enfoques de coreografía como de orquestación, y Event Sourcing, que te permite almacenar eventos como la fuente única de verdad. Estas prácticas te ayudaron a comprender cómo diseñar arquitecturas que sean flexibles, auditables y capaces de manejar grandes volúmenes de datos en tiempo real.

Con este conocimiento, estás mejor preparado para diseñar y desarrollar sistemas distribuidos que aprovechen al máximo los beneficios de la comunicación asíncrona y la orientación a eventos. Trabajar con eventos no solo mejora la escalabilidad y flexibilidad del sistema, sino que también incrementa la tolerancia a fallos, permitiendo que los servicios continúen funcionando incluso si uno de ellos falla temporalmente, gracias a la capacidad de almacenar y reprocesar eventos.

Referencias

- Coder, D. (2023). *Fundamentos de Apache.* Medium. https://medium.com/@diego.coder/introducci%C3%B3n-a-apache-kafka-d1118be9d632
- Coder, D. (2023). *Introducción a la arquitectura orientada a eventos.* Medium. https://medium.com/@diego.coder/introducci%C3%B3n-a-la-arquitectura-orientada-a-eventos-a532c71c9945
- Kafka. (s.f.). *Kafka Introduction.* Apache Kafka. https://kafka.apache.org/intro
- Microsoft. (s.f.). Patrón Event Sourcing. Learn.microsoft. https://learn.microsoft.com/es-es/azure/architecture/patterns/event-sourcing
- Richardson, C.(s.f.). *Pattern: Saga.* Microservices. https://microservices.io/patterns/data/saga.html
- Richardson, C.(s.f.). *Managing data consistency in a microservice architecture using Sagas - Implementing a choreography-based saga.* Microservices. https://microservices.io/post/sagas/2019/08/15/developing-sagas-part-3.html
- Richardson, C.(s.f.). *Pattern: Saga.* Microservices. https://microservices.io/patterns/data/saga.html
- Vichugovae, A. (2022). *Brokers Apache Kafka vs JMS: 3 diferencias principales.* Bigdataschool. https://bigdataschool.ru/blog/kafka-vs-jms-brokers.html

Lecturas de la semana

- Kafka. (s.f.). Kafka Introduction. Apache Kafka. https://kafka.apache.org/intro
- Richardson, C. (s.f.). *Pattern: Saga.* Microservices. https://microservices.io/patterns/data/saga.html
- Spring. (s.f.). *Spring framework reference documentation.* Spring. https://docs.spring.io/spring-framework/reference/integration/jms.html

Apuntes



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC
La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.